

Persist backtest out-of-sample predictions under their own model versions

Context

`evaluation/checkpoint12.py` runs the real backtest — roughly 66,610 rows per target across 129 monthly folds, 2015 to 2026 — and writes a report and a model card to files. It does not persist the per-row predictions. So the strongest evidence that the model works exists only as text in a markdown file and cannot be displayed, queried, or charted.

Every prediction the backtest makes is out-of-sample: the model that produced it was trained only on earlier dates and had not seen that match. That is exactly what should be shown on a dashboard, because it is an honest estimate of real-world performance rather than the model grading its own homework.

This card makes those predictions queryable.

Tasks

1. Report how backtest predictions are currently held in memory: the function that produces them, the dataframe columns available, and whether `match_id`, `player_id`, `prop_type`, and the three quantile values can all be recovered per row. If any of those is missing, say so before writing any persistence code.
2. Choose and state a `modelversion` naming scheme that cannot collide with live predictions. Use an explicit suffix such as `gbm_v1_aces_backtest`. Write the chosen scheme into the report so it is recorded somewhere other than the code.
3. Persist the backtest's out-of-sample predictions using the existing `upsert_predictions` from `models/baseline.py`. Do not write a second writer. If the existing writer cannot accept the backtest's frame shape, adapt the frame, not the writer.
4. Respect the interval CHECK from migration 013: `lowerci` must be at least 0 and must satisfy `lowerci <= prediction <= upperci`. The quantile models can emit crossed quantiles on some rows. Report how many rows required clamping and what you clamped them to — do not silently repair them.
5. Record which fold each prediction came from, along with the train-window end date for that fold. Without it there is no way to confirm after the fact that a prediction was genuinely out-of-sample. If this needs a new column or table, add it in the next numbered migration.
6. Report the persisted counts grouped by `modelversion`, and the earliest and latest `event_date` covered. Confirm the count is in the expected range given roughly 66,610 rows per target, and if it is materially different, explain the gap rather than accepting it.
7. Prove the out-of-sample property on real persisted data. For three sampled predictions, show the fold's train-window end date and the match's `event_date`, and confirm the event date is later. Paste the actual rows.

Do not

Do not use random train/test splits anywhere. Do not overwrite or reuse a `modelversion` that already exists in the table. Do not write a new prediction writer. Do not change model hyperparameters or features in this card — persistence only.

Report

Report which items you completed and explicitly list any you did not do and why. Do not silently skip items.

What to look for

Item 7 is the whole point of the card and the easiest to fake. "Out-of-sample" is a claim about the order of two dates, and it is trivially checkable against the persisted rows. Demand the actual dates, not a statement that the property holds.

Item 4 matters because quantile regression fits each quantile separately, so nothing forces the 10th percentile to come in below the 90th on every row. Where they cross, the interval is meaningless and the database will reject it. How many rows needed clamping is a quality signal worth knowing: a handful is normal, a large fraction means the quantile models are poorly fit.